Ganita Manjari | Grade 9 — Solutions

Chapter 11: Algorithms & Data Structures • End-of-Chapter Exercises (Page 50)

QUESTION 1: IMPROVED EUCLID'S ALGORITHM (GCD BY MODULO)

Method Note

The **improved Euclid's algorithm** uses remainder (modulo operation `a mod b`) instead of repeated subtraction: `gcd(a, b) = gcd(b, a mod b)` until remainder is 0.

(i) Compute gcd(375, 825)

```
• 825 mod 375 = 75   → gcd(375, 75)
• 375 mod 75  = 0    → remainder is 0!
GCD = 75
```

(ii) Compute gcd(51000, 81000)

```
• 81000 mod 51000 = 30000  → gcd(51000, 30000)
• 51000 mod 30000 = 21000  → gcd(30000, 21000)
• 30000 mod 21000 = 9000   → gcd(21000, 9000)
• 21000 mod 9000  = 3000   → gcd(9000, 3000)
• 9000 mod 3000   = 0    → remainder is 0!
GCD = 3000
```

(iii) Compute gcd(1789287, 237656)

```
• 1789287 mod 237656 = 125695  → gcd(237656, 125695)
• 237656 mod 125695 = 111961  → gcd(125695, 111961)
• 125695 mod 111961 = 13734   → gcd(111961, 13734)
• 111961 mod 13734 = 2089    → gcd(13734, 2089)
• 13734 mod 2089 = 1200    → gcd(2089, 1200)
• 2089 mod 1200 = 889     → gcd(1200, 889)
• 1200 mod 889 = 311     → gcd(889, 311)
• 889 mod 311 = 267     → gcd(311, 267)
• 311 mod 267 = 44     → gcd(267, 44)
• 267 mod 44 = 3      → gcd(44, 3)
• 44 mod 3 = 2      → gcd(3, 2)
• 3 mod 2 = 1      → gcd(2, 1)
• 2 mod 1 = 0      → remainder is 0!
GCD = 1
```

Question 2

Assume m ≥ n. Verify that d divides m and n if and only if d divides n and (m mod n).

Easy Explanation:

By definition of division, we can write: `m = q × n + (m mod n)`, where q is quotient.

- **Forward direction (⇒):** If d divides both m and n, then `(m mod n) = m - q × n` is a difference of two multiples of d, so d must also divide `(m mod n)`.
- **Reverse direction (⇐):** If d divides both n and `(m mod n)`, then `m = q × n + (m mod n)` is a sum of two multiples of d, so d must also divide `m`.

1 (iv) Compute gcd(2587392, 157656)

```
• 2587392 mod 157656 = 64896  → gcd(157656, 64896)
• 157656 mod 64896 = 27864  → gcd(64896, 27864)
• 64896 mod 27864 = 9168   → gcd(27864, 9168)
• 27864 mod 9168 = 360    → gcd(9168, 360)
• 9168 mod 360 = 168    → gcd(360, 168)
• 360 mod 168 = 24    → gcd(168, 24)
• 168 mod 24 = 0    → remainder is 0!
GCD = 24
```

QUESTION 3, 4 & 5: PRIME ALGORITHMS & FACTORISATION

Question 3

Write an algorithm prime(n) to check if n is prime.

Easy Steps:

1. If $n \leq 1$, return `False` (not prime).
2. Get the list of divisors of n using `divisors(n)` up to √n.
3. If the length of divisors list is exactly 1 (which is just the number 1 itself), then n is **Prime** (returns `True`). Otherwise, return `False`.

Question 4

Write an algorithm primedivisors(n) to compute prime divisors of n.

Easy Steps:

1. Compute all divisors of n and store them in a list `D = divisors(n)`.
2. Initialize an empty list `prime_list = []`.
3. For each element d in D: check if `prime(d)` is `True`.
4. If `True`, add d to `prime_list`. Return `prime_list`.

Question 5

Prime Factorisation & Representation

(i) Representation of 180 = $2^2 \times 3^2 \times 5^1$:

- We can represent prime factorisation as a list of pairs or dictionary of `(prime_base, exponent)`.
- For 180: `[(2, 2), (3, 2), (5, 1)]`.

(ii) How to compare prime factorisations of two numbers to find GCD?

- Find prime factorisations for both numbers.
- For every common prime factor, pick the **smaller exponent**.
- Multiply these common prime factors raised to their lower exponents to get the GCD!